CITS3402 - High Performance Computing
Assignment 1: Report

The University of Western Australia

James Oakey (22709404) Brent Tankink (23842541)

Semester 2, 2026

1 Introduction

This document outlines how the puzzling results of the Birthday Paradox can be exploited in order to brute force hash collisions. The goal is to alter the 'nonce' values in two PDF files, each with differing contents, in such a way that the resultant hash is identical.

This collision is produced through the combination of nonce variation and the Birthday Attack principle, which proves that collision probability increases rapidly with less trials than is intuitively expected.

2 Birthday-Attack Algorithm

The implemented search strategy uses a Hash-Table data structure to store and search through the calculated hashes. Rather than comparing every possible nonce produced from one PDF file to the other, each calculated (hash+file) combination is checked against those already in the table. Essentially, rather than checking whether a match occurs with one specific hash, the program checks whether a match occurs between any two hashes.

The `toy_hash()` algorithm produces 48-bit hashes, meaning there are 2^{48} possible hashes. Using the Birthday Paradox, the expected number of trials for a 50% collision probability follows $2^{\sqrt{48}} = 2^{24}$, meaning only 16-17 million calculations are required to achieve this 50% probability. To find an upper bound for the calculation process, the following formula can be used: $P(\text{No Collision}) \approx e^{-\frac{n^2}{2H}}$, where n is the number of trials required to achieve P probability of no collision when there are H possible outputs. For a 99% chance of producing a collision:

$$0.01 = e^{-\frac{n^2}{2\times 2^{48}}} \quad \rightarrow \quad n \approx 51,000,000$$

For the stated reason above, the maximum number of iterations is capped at 50,000,000, as this will give a very high probability of collision and won't require a huge initial memory allocation.

To store generated hashes, a hash table has been implemented using linear probing. Each entry, `HashEntry`, contains a 64-bit tag and a 64-bit nonce, making each entry 16 bytes. The `tag` attribute combines the PDF file number (1 or 2) and the 48-bit file hash, making efficient use of space and allowing the program to distinguish hashes from PDF1 and PDF2. This table is initialised using `calloc`, making each entry 0. Whenever a new hash is generated, the lowest 27 bits of the hash value are used to calculate the insertion index in the hash table. If slot is already allocated, linear probing is performed to find the next free slot. The produced hashes and file identifiers are then checked against previous ones, allowing the collision detection between the two PDFs.

The hash table created contains 2^{27} 16-byte entries, meaning the table uses just over 2GB.

3 Parallelisation with OpenMP

The serial implementation works by assigning a different nonce to each PDF in every loop iteration, then calculating the hash for both files. These hashes are then checked against the hash table, if either of them match a hash which was produced by the other PDF, then a collision is found and the collision pair is returned (if no collision is found then the two are inserted in the table). The table insertion is done through a function named `insert_or_match()`, which performs the lookup and probing outlined in the birthday attack section. The basic process is as follows:

| 1. Set and insert nonces into PDF 1 and 2
| 2. Calculate hash of each PDF
| 3. Check hash table
↪ 3a. If match: save nonces and stop
↪ 3b. If not: insert hashes into table and continue

The parallel implementation works by splitting the nonce generation and the PDF hashing across multiple threads in a `parallel` region. The main `for` loop is split into a parallel region with `schedule(static)`, as each iteration performs a similar amount of work.

Each thread creates its own copies of the two PDFs so that the `set_nonce()` function doesn't continuously overwrite each thread's contents (which could cause incorrect hash calculations), eliminating a race condition. The hash table is shared between all threads, so the lookup and insertion operations are isolated inside a critical region. This way, there will be no data races occurring in the shared table. Without this critical region, two threads could simultaneously try to insert a hash value in the same hash table location, making the table state inconsistent. The critical region ensures that only 1 thread accesses the table at a time.

The computationally expensive operations are placed outside of this critical region, with those being `set_nonce()` and `toy_hash()`. This minimises the size of the critical region and the time that threads are prevented from running concurrently. The `found` variable is also shared, which in the serial implementation checks whether a collision has occurred. In the parallel code, this variable needs to be synchronised so that all threads know when a collision has occurred. Each iteration `atomic read`'s the `found` variable before entering the table checker, which makes sure that any collision is detected. Upon entering the critical region, this `found` value is checked again, as there is a possibility that a collision has occurred within the small time frame between the initial check and the critical region.

There are a couple of trade-offs with this approach, and these are discussed further in the following section.

4 Memory Usage & Trade-Offs

As stated in the Birthday-attack algorithm, the hash table size is set as $2^{27} \times 16$ Bytes $\approx$ 2GB bytes, capping the memory at just over 2 gigabytes. This was picked to ensure that the maximum 50,000,000 trials (for each PDF) can be stored while having some space remaining to minimise hash table probing collisions. While dynamic hash table size increasing would have been more memory efficient, this would have created a much longer running time, hence it was decided to allocate the large initial hash table.

A particular trade-off made in the use of a critical region instead of a multi- partitioned table as the critical region approach has it's speed limited by the shared table. Increasing the thread count will increase the concurrent nonce and hash calculations, but the critical region means only one thread can access the table at a time, bottlenecking the program. An alternative to this could be a partitioned hash table (mentioned in the task brief), where different threads can access different partitions side by side. The partitioned approach would have produced a

speed-up, however it would likely have been much more difficult to implement. The possibility for increased speed was traded for simplicity.

5 Performance Evaluation

5.1 Methodology

5.2 Speedup and scalability

5.3 Discussion

6 Conclusion

A Reproducing the results